Centro Paula Souza

Fatec Jahu

INTRODUÇÃO AO MONGODB

BANCO DE DADOS NÃO RELACIONAIS

CST em Desenvolvimento de Software Multiplataforma

PROF. Me. TIAGO A. SILVA

SOBRE O MONGODB

- Breve Histórico:
  - Criado em 2007 pela 10gen (hoje MongoDB Inc.).
  - Lançado como código aberto em 2009.
  - Popularizou-se como um banco de dados NoSQL escalável e flexível.

SOBRE O MONGODB

- Principais Características :
  - Modelo de Dados: Armazena dados em documentos JSON (BSON).
  - Schema Flexível: Não exige esquema fixo como bancos relacionais.
  - Escalabilidade Horizontal: Suporte a sharding para distribuir dados.
  - Alta Performance: Índices e cache embutidos para leitura/escrita rápida.
  - Alta Disponibilidade: Replica sets garantem redundância e failover automático.

COMPARATIVO ENTRE SQL E NOSQL

Um breve comparativo sobre nomenclaturas e correlação de conceitos

CONCEITOS DE SQL E NOSQL

Característica	Banco de Dados Relacional (SQL)	MongoDB (NoSQL)
Modelo de Dados	Baseado em **tabelas** (linhas e colunas)	Baseado em **documentos** (JSON/BSON)
Esquema	**Rígido**, exige definição prévia de tabelas e colunas	**Flexível**, permite diferentes estruturas em um mesmo conjunto de dados
Linguagem de Consulta	**SQL (Structured Query Language)**	**MongoDB Query Language (MQL)** baseada em JSON
Relacionamentos	Relacionamentos fortes com **chaves primárias e estrangeiras**	Relacionamentos menos estruturados, geralmente embutidos nos documentos

CONCEITOS DE SQL E NOSQL

Característica	Banco de Dados Relacional (SQL)	MongoDB (NoSQL)
Escalabilidade	**Vertical** (mais CPU/RAM no mesmo servidor)	**Horizontal** (distribuição de dados entre múltiplos servidores via Sharding)
Performance	Pode ser mais lento para grandes volumes de dados devido a joins complexos	Mais rápido para leituras/escritas frequentes, pois evita joins
Transações	**ACID (Atomicidade, Consistência, Isolamento e Durabilidade)**	Suporte a transações multi-documento, mas com foco em **eventual consistency**
Armazenamento de Dados	Estruturado e normalizado (divisão em múltiplas tabelas)	Desnormalizado, geralmente armazenando tudo em um único documento
Índices	Índices primários e secundários	Suporte a índices, incluindo índices geoespaciais e texto
Casos de Uso	Sistemas financeiros, ERP, CRM, e-commerce tradicional	Big Data, IoT, aplicações web escaláveis, análise de logs, redes sociais

CONCEITOS DE SQL E NOSQL

Conceito SQL	Equivalente no MongoDB	Descrição
Banco de Dados	**Database**	Conjunto de tabelas no SQL ou de coleções no MongoDB.
Tabela	**Coleção (Collection)**	Agrupamento de registros (SQL) ou documentos (MongoDB).
Linha / Registro	**Documento (Document)**	No SQL, uma linha contém valores em colunas; no MongoDB, um documento é um objeto JSON armazenado na coleção.
Coluna	**Campo (Field)**	Representa um atributo dentro do documento (como uma chave em um JSON).
Tupla	**Documento (Document)**	Em SQL, uma tupla é um conjunto de valores em uma linha; em MongoDB, o documento armazena esses valores.

CONCEITOS DE SQL E NOSQL

Conceito SQL	Equivalente no MongoDB	Descrição
Chave Primária	**_id**	Identificador único automático em cada documento do MongoDB.
Chave Estrangeira	**Referência entre documentos**	No SQL, a chave estrangeira relaciona tabelas; no MongoDB, podemos armazenar um campo com o _id de outro documento ou usar documentos embutidos.
Índice	**Índice (Index)**	Ambos utilizam índices para acelerar consultas. Em MongoDB, índices são criados com createIndex().
View	**Consulta com projeção**	No SQL, views são consultas salvas. No MongoDB, podemos simular isso com find() e projeções.

CONCEITOS DE SQL E NOSQL

Conceito SQL	Equivalente no MongoDB	Descrição
JOIN	**`$lookup` (Aggregation Framework)**	Em SQL, usamos JOIN para unir tabelas; no MongoDB, usamos agregação com `$lookup` para combinar documentos de coleções diferentes.
Stored Procedure	**Scripts no Aplicativo**	MongoDB não tem stored procedures, mas podemos usar scripts JavaScript no shell ou código na aplicação.
Transação	**Sessões e transações**	MongoDB suporta transações multi-documento, mas prioriza consistência eventual.

COMANDOS BÁSICOS DE INTERAÇÃO COM MONGODB

Conexão com o MongoDB e interação com documentos

COMANDOS BÁSICOS DO MONGODB

Comando	Ação
`mongo`	Inicia o shell do MongoDB e conecta ao servidor padrão (localhost:27017).
`use meuBanco`	Se meuBanco não existir, ele será criado assim que um documento for inserido.
`show dbs`	Exibe os bancos disponíveis. Se um banco estiver vazio, ele não será listado.
`show collections`	Mostra todas as coleções em um banco
`use meuBanco;` `db.dropDatabase()`	Seleciona e em seguida exclui o banco de dados

COMANDOS BÁSICOS DO MONGODB: INSERIR DOCUMENTOS

Comando	Ação
`db.createCollection("usuarios");`	MongoDB cria coleções automaticamente quando documentos são inseridos.
`db.usuarios.insertOne( { nome: "Alice", idade: 25, cidade: "São Paulo" } );`	Inserção de um único documento
`db.usuarios.insertMany([ { nome: "Bob", idade: 30, cidade: "Rio de Janeiro" }, { nome: "Carlos", idade: 22, cidade: "Belo Horizonte" } ]);`	Inserção de múltiplos documentos

COMANDOS BÁSICOS DO MONGODB: FILTRAR DOCUMENTOS

Comando	Ação
`db.usuarios.find({ idade: 25 }).pretty();`	Filtrar documentos (exemplo: encontrar usuários com idade 25)

`db.usuarios.find(`
`  { cidade: "São Paulo" },`
`  { nome: 1, _id: 0 }`
`);`
	Filtrar e exibir apenas alguns campos. Retorna apenas o campo **`nome`**, excluindo **`_id`**.

1 e 0 definem quais campos irão aparecer na consulta. Campos com valor 0 (false) não aparecem.

COMANDOS BÁSICOS DO MONGODB: ATUALIZAR DOCUMENTOS

Comando	Ação
`db.usuarios.updateOne( { nome: "Alice" }, { $set: { idade: 26 } } );`	Atualizar um único documento
`db.usuarios.updateMany( { cidade: "São Paulo" }, { $set: { estado: "SP" } } );`	Atualizar múltiplos documentos. Todos os usuários de São Paulo agora terão o campo estado: "SP".
`db.usuarios.updateOne( { nome: "Alice" }, { $push: { hobbies: "leitura" } } );`	Adicionar um novo valor a um array dentro do documento

Os operadores `$set` e `$push` serão estudados em detalhes posteriormente.

COMANDOS BÁSICOS DO MONGODB: REMOVER DOCUMENTOS

Comando	Ação
`db.usuarios.deleteOne( { nome: "Carlos" } );`	Remover um único documento
`db.usuarios.deleteMany( { idade: { $lt: 25 } } );`	Remove todos os usuários com menos de 25 anos.

O operador `$lt` será estudado em detalhes posteriormente.

COMPARAÇÃO ENTRE DADOS NO SQL E NOSQL

Um breve exemplo

DADOS EM TABELAS RELACIONAIS

- Tome como base um modelo relacional com as seguintes tabelas e registros:

id_cliente	nome	email
1	João Silva	joao@email.com

id_pedido	id_cliente	produto	valor
101	1	Notebook	3000
102	1	Smartphone	1500

INSERÇÃO DE DADOS NO MODELO RELACIONAL

- Para inserir os registros, deveríamos:

```sql
INSERT INTO clientes (id_cliente, nome, email) VALUES (1, 'João Silva', 'joao@email.com');

INSERT INTO pedidos (id_pedido, id_cliente, produto, valor) VALUES
(101, 1, 'Notebook', 3000),
(102, 1, 'Smartphone', 1500);
```

MODELO NOSQL (MONGODB) COM EMBEDDED DOCUMENT

- ✓ Em MongoDB, podemos armazenar os pedidos dentro do próprio documento do cliente, evitando a necessidade de JOIN

```
1  db.clientes.insertOne({
2      _id: 1,
3      nome: "João Silva",
4      email: "joao@email.com",
5      pedidos: [
6          { id_pedido: 101, produto: "Notebook", valor: 3000 },
7          { id_pedido: 102, produto: "Smartphone", valor: 1500 }
8      ]
9  })
```

EXERCÍCIO

- ✓ Converta seus banco de dados SQL para o modelo NOSQL usando o mongodb

OBRIGADO!

- Encontre este **material on-line** em:
  - Slides: Plataforma Microsoft Teams
- Em caso de **dúvidas**, entre em contato:
  - **Prof. Tiago**: tiago.silva238@fatec.sp.gov.br